ADV TP1: What I Understood

Advanced Programming for Data Science

HEM Bellyday

Department of Applied Mathematics and Statistics (AMS)
Institute of Technology of Cambodia (ITC)
Lecturer: Mr. VANN Visal

Lecture of October 13, 2026

Outline

1. Part 1: Building Blocks
2. Part 2: Clean Code
3. Part 3: PEP 8
4. Part 4: Debugging

Part 1: Building Blocks (1/2)

1. What is a function?

This is like a math function: it has something like x or y in it when we define it. We use it so we don't do a task again and again; we just use this function and finish. (Write once, use quickly.)

2. What is a variable?

A variable is like a math definition: it refers to a value or object when we want to define it.

3. What is a class?

A class is like a function, but it has a bunch of functions inside of it.

Part 1: Building Blocks (2/2)

4. What is a method?

A method is a function that makes something work, or does an action, inside a class. Example on the board: calculate score.

5. What is a module?

A module is like a `.py` file: we write functions inside it and call them to use in the future in another module.

6. What is a package?

A bunch of modules stuck together. When we install a package, that is a bunch of modules ready to use.

7. What is a constant?

It refers to a value that doesn't change, even in future use.

Part 2: Clean Code

Before clean code

The teacher asked what is something in class we don't understand, and I said PEP 8.

What is clean code?

Clean code refers to a file or code lines written so that not only the user can understand them, but others can too.

Part 2: Clean Code Practices

10. **Descriptive name:** name the variable so it can be understood quickly, and keep it short. Example: a function to start a moto engine; there are many names we could use, but use one name that other functions in the moto can't use.
11. **Comment:** # that explains code in a single line.
12. **Docstring:** used to explain a whole function or class so others can understand it.
13. **Naming values:** name values that can't be changed (even in the future they don't change), so it is easy and we don't have to find that value in whatever file it is in.
14. **Functions:** a function needs to be clear and reasonable to use.

Part 3: What is PEP 8?

PEP 8

PEP 8 is created by Python itself, just to make code structure more clean and reasonable, and it is also a rule to follow from Python.

What	Style	Example
Variables, functions, methods	`snake_case`	`student_score`
Constants	`UPPER_CASE`	`PASS_MARK`
Classes	`PascalCase`	`Student`

Part 3: PEP 8 Rules in Simple Words

- **Indentation:** 4 spaces for each level.
- **Line length:** at most 79 characters for code, 72 for comments and docstrings.
- **Imports:** put them at the top. First Python's built-in modules, then installed libraries, then my own files, with a blank line between each group. Don't use `from x import *`.
- **Spaces:** one space around = and ==, but no extra space just inside brackets or before the (of a call.
- **Commas and colons:** a space after a comma, like `[60, 80]`, but no spaces in a slice, like `scores[1:3]`.
- **Blank lines:** 2 between top-level functions or classes, 1 between methods inside a class.
- **Long lines:** wrap them inside parentheses, and break the line before an operator like + or and.
- **Quotes:** choose ' or " and keep using the same one. Docstrings use """.

Part 3: PEP 8 Example

Not following PEP 8

```
passmark=50
def HasPassed( s ):
  return s>=passmark
print (HasPassed(75))
```

Following PEP 8

```
PASS_MARK = 50


def has_passed(score):
    return score >= PASS_MARK


print(has_passed(75))
```

Part 4: Debugging

What is debugging?

Fixing errors, of course. But sometimes the error shown is not clear or reasonable, so we have to check further in the code file.

Tools the teacher recommended

The teacher recommended 2 old-school tools. Even before AI existed, these are what old programmers used. (I forgot the names.)

Part 4: How I Debug

1. Print the suspicious code line, so we can clearly see the error and know what to fix.
2. Don't put 5 million lines of code in 1 file. Divide them and use them as imports. When there is an error, it is easy to find in the file structure.

That's all I learned from that day.